

Activity 3 — Exploratory Data Analysis (EDA)

What you'll learn

In Activity 2 you **collected** data. Now you **analyze** it — the real job of a data analyst. You'll take the **tourism table** from `car_data.html` and explore it: check its quality, summarize it, compare groups, look for relationships, and visualize the whole thing.

Before you start

- A **Google account** to use Google Colab.
- **car_data.html** loaded into Colab — upload it (folder icon → upload) or use the optional download cell in Step 1.

Reminder: this is **simulated sample data** for training. The numbers are realistic but invented, so any pattern or recommendation here demonstrates the **method** — it is not real advice about CAR tourism.

Run each cell with **Shift + Enter**, top to bottom.

Step 1 — Setup and load the scraped data

Upload **car_data.html** into Colab first. This cell re-scrapes the tourism table (the 3rd table) so this notebook works on its own.

[COPY THIS CODE](#)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

# Scrape the tourism table (3rd table in the file)
df = pd.read_html('car_data.html')[2]

print(f"Loaded {len(df)} tourism sites with {df.shape[1]} columns.")
print("Columns:", list(df.columns))
display(df.head())
```

Can't upload? Easier alternative — run this cell first to download the file, then run Step 1:

DOWNLOAD INSTEAD OF UPLOAD

```
import urllib.request
urllib.request.urlretrieve(
    "https://datasciencebaguio-production.up.railway.app/datasets/car_data",
    "car_data.html")
print("Downloaded car_data.html")
```

You should see: "Loaded **40** tourism sites with **9** columns." and the column list.

Step 2 — Know your columns

A quick look at data types and a statistical summary of the numeric columns.

COPY THIS CODE

```
print("Data types:")
print(df.dtypes)

print("\nSummary of numeric columns:")
display(df[['Annual_Visitors', 'Entry_Fee_PHP', 'Rating']].describe().round(2))
```

You should see: the dtype of each column and a describe() table (count = 40).

Step 3 — Data quality check

Real data is messy. Check for missing values and duplicate rows before trusting it.

COPY THIS CODE

```
missing = df.isnull().sum()
print("Missing values per column:")
if missing.sum() == 0:
    print("  None. Clean data.")
else:
    print(missing[missing > 0])

duplicates = df.duplicated().sum()
print(f"\nDuplicate rows: {duplicates}")
if duplicates == 0:
    print("  None.")
```

You should see: "None. Clean data." and "Duplicate rows: 0".

Step 4 — Count things (categorical breakdown)

How are the sites split across provinces and types? `value_counts` is your friend.

COPY THIS CODE

```
print("Sites by Province:")
for prov, n in df['Province'].value_counts().items():
    print(f"  {prov}: {n} ({n/len(df)*100:.0f}%)")

print("\nSites by Type:")
for t, n in df['Type'].value_counts().items():
    print(f"  {t}: {n} ({n/len(df)*100:.0f}%)")
```

You should see: counts and percentages by province and by type.

Step 5 — Key metrics

The headline numbers a manager would ask for.

COPY THIS CODE

```
total_visitors = df['Annual_Visitors'].sum()
avg_visitors = df['Annual_Visitors'].mean()
avg_rating = df['Rating'].mean()
free_sites = (df['Entry_Fee_PHP'] == 0).sum()
paid_sites = (df['Entry_Fee_PHP'] > 0).sum()

print(f"Total annual visitors: {total_visitors:,}")
print(f"Average visitors per site: {avg_visitors:,.0f}")
print(f"Average rating: {avg_rating:.2f} out of 5")
print(f"Free-entry sites: {free_sites} | Paid sites: {paid_sites}")
```

You should see: totals, averages, and a free-vs-paid count.

Step 6 — Compare groups

Grouping is the heart of analysis. Here we rank provinces and types by their numbers.

COPY THIS CODE

```
print("PROVINCE PERFORMANCE (by total visitors):")
province_stats = df.groupby('Province').agg(
    Total_Visitors=('Annual_Visitors', 'sum'),
    Avg_Visitors=('Annual_Visitors', 'mean'),
    Avg_Rating=('Rating', 'mean'),
    Site_Count=('Site ID', 'count')
).round(1).sort_values('Total_Visitors', ascending=False)
display(province_stats)

print("\nTOURISM TYPE PERFORMANCE (by total visitors):")
type_stats = df.groupby('Type').agg(
    Total_Visitors=('Annual_Visitors', 'sum'),
    Avg_Visitors=('Annual_Visitors', 'mean'),
    Avg_Rating=('Rating', 'mean'),
    Avg_Fee=('Entry_Fee_PHP', 'mean')
).round(1).sort_values('Total_Visitors', ascending=False)
display(type_stats)
```

You should see: two ranked tables — by province and by type.

Step 7 — Top and bottom performers

COPY THIS CODE

```
print("TOP 5 MOST VISITED:")
top = df.nlargest(5, 'Annual_Visitors')[['Site Name', 'Province', 'Annual_Visitors',
'Rating']]
display(top)

print("BOTTOM 5 (fewest visitors):")
bottom = df.nsmallest(5, 'Annual_Visitors')[['Site Name', 'Province', 'Annual_Visitors',
'Rating']]
display(bottom)
```

You should see: a top-5 and a bottom-5 table.

Step 8 — Correlation: do numbers move together?

Correlation ranges from -1 to $+1$. Near 0 means no relationship. **Because this data is randomly generated, these correlations will be near zero and mean nothing** — that is itself a useful lesson.

COPY THIS CODE

```
corr = df[['Annual_Visitors', 'Entry_Fee_PHP', 'Rating']].corr()
print("Correlation matrix:")
display(corr.round(3))

plt.figure(figsize=(6, 5))
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, fmt='.3f',
            square=True, linewidths=1)
plt.title('Correlation Between Numeric Variables')
plt.tight_layout()
plt.show()
```

You should see: a correlation table and a heatmap (values near zero).

Step 9 — The visual dashboard

Six clean charts summarizing the whole dataset.

COPY THIS CODE


```
sns.boxplot(data=df, x='Type', y='Entry_Fee_PHP', ax=axes[1, 2])
axes[1, 2].set_title('Entry Fee by Type')
axes[1, 2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

You should see: a 6-chart dashboard (h-bar, pie, histogram, scatter with colorbar, bar, boxplot).

Step 10 — Turning analysis into findings

Good analysts don't just make charts — they say what the numbers mean. This computes findings straight from the data.

Big caveat: the data is simulated, so these "findings" demonstrate **how** analysis leads to decisions. They are not real advice about CAR tourism.

COPY THIS CODE


```

top_prov = province_stats.index[0]
low_prov = province_stats.index[-1]
gap = province_stats.loc[top_prov, 'Total_Visitors'] / province_stats.loc[low_prov,
'Total_Visitors']

best_type = type_stats.index[0]
free_avg = df[df['Entry_Fee_PHP'] == 0]['Annual_Visitors'].mean()
paid_avg = df[df['Entry_Fee_PHP'] > 0]['Annual_Visitors'].mean()

print("FINDINGS (from the simulated data):")
print(f"1. {top_prov} leads in total visitors; {low_prov} is lowest. Gap is about
{gap:.1f}x.")
print(f"2. '{best_type}' sites draw the most visitors overall.")
print(f"3. Free sites average {free_avg:,.0f} visitors; paid sites average
{paid_avg:,.0f}.")
print(f"4. Average rating across all sites is {df['Rating'].mean():.2f} out of 5.")

print("\nHOW THIS WOULD DRIVE DECISIONS (illustrative only):")
print("- A real analyst would ask why the lowest province lags, then check infrastructure
or access.")
print("- They'd promote high-rated but low-traffic sites.")
print("- They'd test whether entry fees actually affect visits, here the data is random,
so we can't.")

print("\nThe skill you practiced: load, check, summarize, group, correlate, visualize,
interpret.")

```

You should see: a numbered list of findings plus how they'd drive decisions.

Tips & common errors

- Run cells **top to bottom** with Shift + Enter.
- **FileNotFoundError in Step 1?** car_data.html isn't loaded — upload it or use the optional download cell first.
- **KeyError on a column?** Column names are case- and spelling-sensitive (e.g. Annual_Visitors, Site ID). Use them exactly.
- The one habit to carry forward: always know whether your data is **real or simulated**.

What's next

You've completed the Week 1 loop: **collect** → **gather** → **understand**. Weeks 2 and 3 go deeper into each part.

PUP Santa Rosa · Data Science & Data Analytics · To save as PDF: press **Ctrl + P** → Destination **Save as PDF** → Save.